# SIMATS ENGINEERING

## ASSESSMENT TOOL 3

**COURSE CODE: MLA03**

**COURSE NAME: Reinforcement learning for Environment and Agent.**

**COURSE OUTCOME COVERED**

**CO5: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)**

**Assessment Tool Weightage: 35% Dr G A. SENTHIL**

***Code of Conduct:***

*I A SHIVA SHANKAR REDDY(192472345) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***A SHIVA SHANKAR REDDY***

***Signature of the student:***

**QUESTIONS: 10** **Total Marks: 50**

**Annexure A**

**Mock Interview**

# 1.Warehouse Robot – Dynamic Programming and Value Iteration

## □ AIM

To develop an optimal navigation policy for a warehouse robot using Dynamic Programming and Value Iteration while avoiding obstacles and minimizing travel distance.

## □ ABSTRACT

Dynamic Programming can solve warehouse navigation by calculating the value of each possible state. Value Iteration repeatedly updates these values using the rewards of possible actions. The robot learns a policy that selects the action with the highest expected future reward, allowing it to reach the destination through an efficient path while avoiding obstacles.

## □ ALGORITHM

1. Create a warehouse grid.
2. Define the starting state and goal state.
3. Mark obstacle cells.
4. Initialize the value of every state.
5. Calculate the value of each possible action.
6. Select the maximum expected value.
7. Update the state values repeatedly.
8. Stop when the values converge.
9. Extract the optimal policy.
10. Follow the policy from the start to the goal.

 **OUTPUT:-**

```
23
24      for r in range(5):
25          for c in range(5):
26
27              if (r, c) == goal or (r, c) in obstacles:
28                  continue
29
30              values = []
31
32              for dr, dc in actions:
33
34                  nr = r + dr
35                  nc = c + dc
36
37                  if nr < 0 or nr >= 5 or nc < 0 or nc >= 5:
38                      continue
39
```

Problems Output Debug Console Terminal Ports ··· Python + ··· |

```
anchigunashekar/OneDrive/Desktop/python-RL/CO5-1.PY
Optimal State Values:
[[-0.43  0.63  1.81  3.12  4.58]
 [ 0.63  0.    3.12  4.58  6.2 ]
 [ 1.81  0.    4.58  6.2   8.  ]
 [ 3.12  4.58  6.2   0.   10.  ]
 [ 4.58  6.2   8.   10.    0.  ]]
```

 **CONCLUSION:-**

Value Iteration calculates the optimal value of each warehouse location and helps the robot select efficient actions. By avoiding obstacle states and maximizing future rewards, the robot can learn a shorter and safer path to the destination.

**2. Recommendation System – Monte Carlo Control**

 **AIM**

To apply Monte Carlo Control to a recommendation system and improve recommendations using customer interaction experiences collected over multiple sessions.

 **ABSTRACT**

Monte Carlo Control learns the value of actions by observing complete user sessions. In a recommendation system, actions represent recommended products or content. After a session ends, the total reward is calculated from user interactions such as clicks, purchases, ratings, and likes. The action-value estimates are updated using these observed returns.

 **ALGORITHM**

1. Initialize the Q-table and recommendation policy.
2. Start a customer session.
3. Observe the customer's state.
4. Select a recommendation using ε-greedy policy.
5. Record the customer interaction.
6. Continue until the session ends.

7. Calculate the total return from the session.
8. Update the Q-value for visited state-action pairs.
9. Improve the recommendation policy using updated Q-values.
10. Repeat for multiple sessions.

**□ OUTPUT:-**

```
 4    returns = {(s, a): [] for s in range(3) for a in range(3)}
 5
 6    epsilon = 0.1
 7
 8    for episode in range(100):
 9
10        episode_data = []
11
12        state = np.random.randint(3)
13
14        for step in range(5):
15
16            if np.random.random() < epsilon:
17                action = np.random.randint(3)
18            else:
19                action = np.argmax(Q[state])
20
```

```
roblems  Output  Debug Console  Terminal  Ports  ···  Python
anchigunashekar/OneDrive/Desktop/python-RL/CO5-2.PY
Recommendation Q-Values:
[[7.   8.32 8.19]
 [7.72 7.33 5.  ]
 [7.78 5.25 7.6 ]]
Learned Recommendation Policy:
[1 0 0]
```

**□ CONCLUSION:-**

Monte Carlo Control improves recommendations by learning from complete customer sessions. As more sessions are observed, the system estimates which recommendations generate higher returns and gradually improves its recommendation policy.

**3. Delivery Robot – SARSA Algorithm**

**□ AIM**

To implement the SARSA algorithm for a delivery robot and develop a policy that adapts to uncertain environmental conditions.

**□ ABSTRACT**

SARSA is an on-policy Reinforcement Learning algorithm that updates the action-value function using the current state, selected action, reward, next state, and next action. For a delivery robot, it can learn safe routes while considering obstacles, traffic, and environmental changes.

**□ ALGORITHM**

1. Initialize the Q-table.
2. Observe the current state.
3. Select an action using ε-greedy policy.

4. Execute the action.
5. Receive the reward and observe the next state.
6. Select the next action using the same policy.
7. Apply the SARSA update equation.
8. Move to the next state and action.
9. Repeat until the delivery is completed.
10. Continue training as the environment changes.

**OUTPUT:-**

```
 2
 3    Q = np.zeros((5, 4))
 4
 5    alpha = 0.1
 6    gamma = 0.9
 7    epsilon = 0.2
 8
 9    for episode in range(100):
10
11        state = np.random.randint(5)
12
13        # Select first action
14        if np.random.random() < epsilon:
15            action = np.random.randint(4)
16        else:
17            action = np.argmax(Q[state])
```

```
Problems  Output  Debug Console  Terminal  Ports  ···  Python

anchigunashekar/OneDrive/Desktop/python-RL/CO5-3.PY
SARSA Q-Table:
[[-0.13 -0.21 -0.16  1.38]
 [ 6.15  1.44 -0.14  2.03]
 [ 4.83  4.82  4.09 15.3 ]
 [ 5.09  6.99  3.84 23.91]
 [18.06  0.    2.25  0.  ]]
```

**CONCLUSION:-**

SARSA learns the value of actions based on the policy that the robot actually follows. Therefore, when environmental conditions change, the robot can update its Q-values through new experiences and adapt its delivery route accordingly.

**4. Robotic Arm – REINFORCE Algorithm**

**AIM**

To apply the REINFORCE policy-gradient algorithm to train a robotic arm to select actions that maximize object manipulation accuracy.

**ABSTRACT**

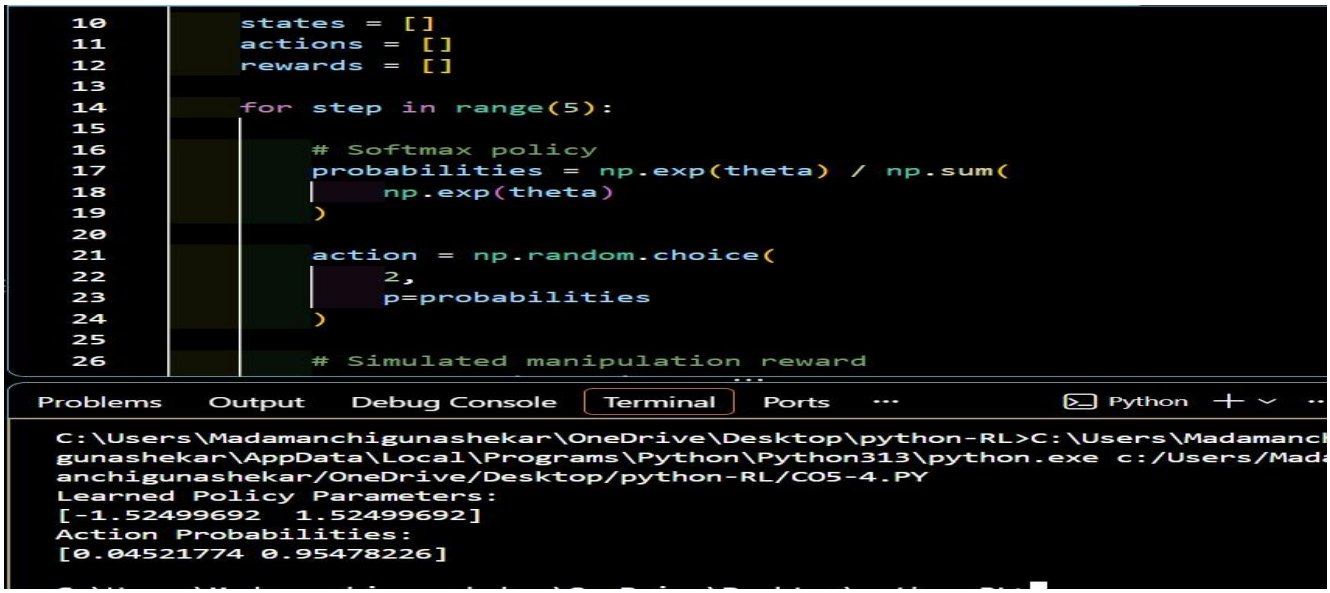
REINFORCE is a policy-gradient algorithm that directly learns a policy instead of a Q-table. The robotic arm selects actions according to probabilities generated by the policy. After completing a task,

the total reward is calculated and the policy is updated so that successful actions become more likely in future attempts.

**□ ALGORITHM**

1. Initialize the policy parameters.
2. Observe the robotic arm's current state.
3. Generate action probabilities using the policy.
4. Select an action.
5. Perform the manipulation action.
6. Store the selected action and reward.
7. Calculate the return after completing the task.
8. Update the policy using the REINFORCE gradient.
9. Increase the probability of successful actions.
10. Repeat until task accuracy improves.

**□ OUTPUT:-**




**□ CONCLUSION:-**

REINFORCE directly optimizes the robotic arm's policy using task rewards. Successful actions receive higher probability during future decisions, allowing the robotic arm to gradually improve object manipulation accuracy.

## 5. Autonomous Vehicle – Q-Learning vs DQN

### □ AIM

To analyze and compare Q-Learning and Deep Q-Networks for training an autonomous vehicle in complex driving environments.

### □ ABSTRACT

Q-Learning stores action values in a Q-table and works effectively when the state space is small and discrete. However, autonomous vehicles have complex states containing images, speed, traffic conditions, distance, and road information. DQN uses a neural network to approximate Q-values and can therefore handle much larger state spaces. DQN generally provides better scalability for complex driving environments.

### □ ALGORITHM

Q-Learning:

1. Initialize Q-table.
2. Observe vehicle state.
3. Select action using ε-greedy policy.
4. Execute action.
5. Receive reward.
6. Update Q-value.
7. Repeat until convergence.

**OUTPUT:-**

```python
import numpy as np

# Simple Q-Learning example
Q = np.zeros((5, 3))

alpha = 0.1
gamma = 0.9
epsilon = 0.1

for episode in range(100):

    state = np.random.randint(5)

    for step in range(10):

        if np.random.random() < epsilon:
            action = np.random.randint(3)
```

Problems 2 Output Debug Console Terminal Ports ··· Python

```
Q-Learning Result:
[[ 3.29 -0.1   0.48]
 [ 0.76 -0.03 10.5 ]
 [17.56  2.25  0.  ]
 [23.95  0.    3.71]
 [18.21  4.87  1.94]]
```

**CONCLUSION**

Q-Learning is simple and converges effectively for small discrete state spaces, but its Q-table becomes impractical as the driving environment becomes more complex. DQN uses a neural network to handle high-dimensional states and is therefore more scalable for autonomous vehicles. Although DQN generally requires more computation and careful training for stable convergence, it is better suited to complex real-world driving environments.

RUBRICS FOR CALCULATION

| Criteria | Weightage | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Technical Knowledge | 25 | Demonstrates strong and accurate subject knowledge with in-depth understanding | Shows good knowledge with minor gaps | Basic knowledge with noticeable errors | Lacks fundamental technical knowledge |
| Problem Solving | 25 | Solves problems logically and applies concepts effectively in real scenarios | Applies concepts with moderate accuracy | Limited problemsolving ability; requires guidance | Unable to solve or apply concepts |
| Analytical Thinking | 15 | Analyzes questions critically and provides wellstructured, logical answers | Provides reasonable analysis with some structure | Superficial or partially structured responses | No analytical thinking demonstrate d |
| Communication | 15 | Communicates clearly, confidently, and professionally with proper terminology | Communicate s well with minor hesitation | Communication lacks clarity or confidence | Unable to communicat e effectively |
| Confidence | 15 | Displays high confidence, positive attitude, and professional behavior throughout | Shows good confidence with minor issues | Low confidence; inconsistent behavior | Lacks confidence and professionali sm |
| Response to Questions | 10 | Responds accurately, promptly, and elaborates with relevant examples | Responds correctly but with limited depth | Partial responses; needs prompting | Unable to respond appropriately |